

Final Technical Report

SentinelLite: A Lightweight SIEM with Rule-Based Detection and ML Anomaly Scoring

Student	Gulraiz Dilawar
ID	F2023376392
Course	Information Security (D1)
Instructor	Muhammad Zunnurain Hussain
Generated	2026-06-17

Abstract

SentinelLite is a lightweight Security Information and Event Management system developed for the Information Security semester project. The system addresses the problem of analyzing heterogeneous security logs in small lab environments where enterprise SIEM deployment may be impractical. SentinelLite ingests Apache access logs, Linux authentication/syslog entries, firewall syslog lines, and Windows Event CSV records. It combines explainable rule-based detections for known attack patterns with Isolation Forest anomaly detection over source-IP behavior windows. The implementation includes a Streamlit dashboard, SQLite persistence, SMTP email alerting, PDF report generation, safe sample logs, and a labeled evaluation set. On the included safe dataset, SentinelLite parsed 49 events, generated 15 meaningful alert groups, detected 3 anomaly windows, and achieved 1.00 group-level precision, recall, and F1 score. The result is a complete academic prototype that demonstrates log management, threat mapping, anomaly scoring, and report-ready security analysis.

Table of Contents

1. Introduction 2. Literature Review 3. System Design 4. Implementation 5. Security Analysis 6. Results and Discussion 7. Conclusion References Appendices

1. Introduction

Security monitoring depends on timely collection and interpretation of logs. In small labs, logs may exist but remain unused because full SIEM platforms require resources, configuration, and operational maturity. SentinelLite focuses on a realistic middle ground: it is small enough to run on a lab machine, but complete enough to demonstrate ingestion, detection, anomaly scoring, reporting, and alerting.

The project objectives are to ingest common logs, apply known-pattern detections, use unsupervised ML for anomaly scoring, provide an interactive dashboard, export evidence, and evaluate performance through labeled test cases.

The scope is intentionally bounded. SentinelLite is not a replacement for a production SOC platform. It does not provide distributed agents, long-term retention tiers, or enterprise correlation rules. Instead, it implements the core concepts required for the Information Security project: log collection, normalization, detection, anomaly scoring, analyst triage, testing, and reporting.

The system is built around safe demonstration data. All included IP addresses are documentation ranges, and attack strings are preserved as inert log text. This supports ethical demonstration without scanning or attacking real systems.

1.1 Objectives and Scope

Objective	Implementation Evidence
Multi-source ingestion	Apache access log, Linux auth/syslog, firewall syslog, Windows Event CSV.
Known-pattern detection	Rules for failed logins, SQL injection, path traversal, scanner agents, 404 enumerations.
Anomaly detection	Isolation Forest over 15-minute source-IP windows.
Analyst workflow	Dashboard filters, investigation expanders, log search, report export, and email digests.
Evaluation	Expected-alert labels and precision/recall/F1 metrics.

2. Literature Review

Background

NIST SP 800-92 emphasizes that log management is a practical security capability covering generation, transmission, storage, analysis, and disposal of log data [1]. SentinelLite narrows this broad discipline into an academic SIEM that focuses on parsing, correlation, triage, and reporting.

Modern SIEM work increasingly combines deterministic rules with anomaly detection. Deterministic rules remain useful for known attack patterns, while anomaly detection helps surface behavior that differs from a local baseline.

For a semester project, the main design challenge is not only detection accuracy; it is also operational completeness. A useful SIEM prototype should show how evidence moves from log source to parser, from parser to alert, and from alert to analyst action.

Machine Learning for Log Anomaly Detection

Isolation Forest is suitable for a lightweight project because it isolates unusual observations rather than requiring a large labeled attack dataset [2]. This aligns with student lab conditions where large labeled enterprise logs are rarely available.

Recent log-analysis systems such as LogLead highlight the importance of loading, enhancing, and evaluating logs through reproducible pipelines [3]. SentinelLite follows the same broad pipeline at a smaller scale: load logs, normalize fields, extract features, detect alerts, and evaluate against a labeled sample.

Recent work on log anomaly detection shows that advanced methods can improve benchmark performance, but practical adoption still depends on explainability, usability, and evaluation design [10], [11].

Deep sequence models and transformer-based approaches can learn richer temporal context, but they require more training data, more compute, and more explanation effort. SentinelLite intentionally uses a classical unsupervised method so the complete system remains feasible on a lab machine.

Threat Mapping and Rule-Based Detection

The MITRE ATT&CK framework provides a vocabulary for mapping detections to adversary behavior. SentinelLite maps brute-force login bursts to T1110 [4], port-scan patterns to T1046 [5], web exploitation probes to T1190 [6], suspicious PowerShell to T1059.001 [7], and log-clearing behavior to

indicator-removal activity [8].

Rule-based detection provides precise explanations for known conditions. In SentinelLite, rules produce evidence strings, recommended responses, severity, confidence, and attack phase so the analyst can understand why an alert exists.

A rule-only system can miss unknown behavior. An ML-only system can produce alerts that are hard to defend. The chosen hybrid approach is therefore a practical compromise: rules provide high interpretability, while anomaly scoring provides broader coverage.

Evaluation Practices

Security testing should report both what was detected and what was missed. The brief explicitly rewards structured test plans, tabulated results, and false-positive/false-negative discussion. SentinelLite therefore includes expected alert labels for the safe sample dataset and computes group-level precision, recall, and F1 score.

The evaluation is intentionally described as sample-set evaluation, not as a universal production claim. This avoids overstating accuracy while still demonstrating that the system is testable and repeatable.

Research Gap

Full SIEM platforms are powerful but heavy for small labs. Research prototypes may focus on algorithms but not on a complete student-demo workflow. SentinelLite fills this gap by combining modest but complete components: multi-source parsing, rules, ML anomaly scoring, SQLite persistence, UI, email alerts, PDF reporting, and evaluation.

The project gap is therefore implementation-oriented: a lightweight, explainable, low-cost SIEM workflow that can be demonstrated live, defended technically, and extended after the semester.

3. System Design

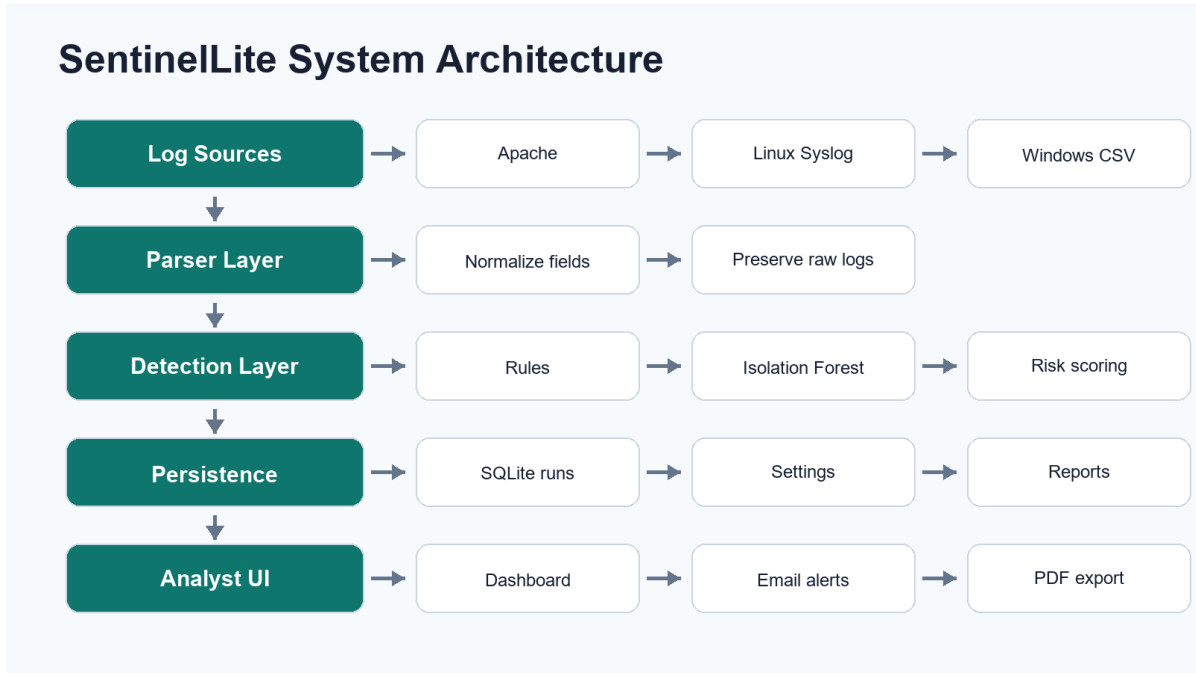


Fig. 1. SentinelLite architecture.

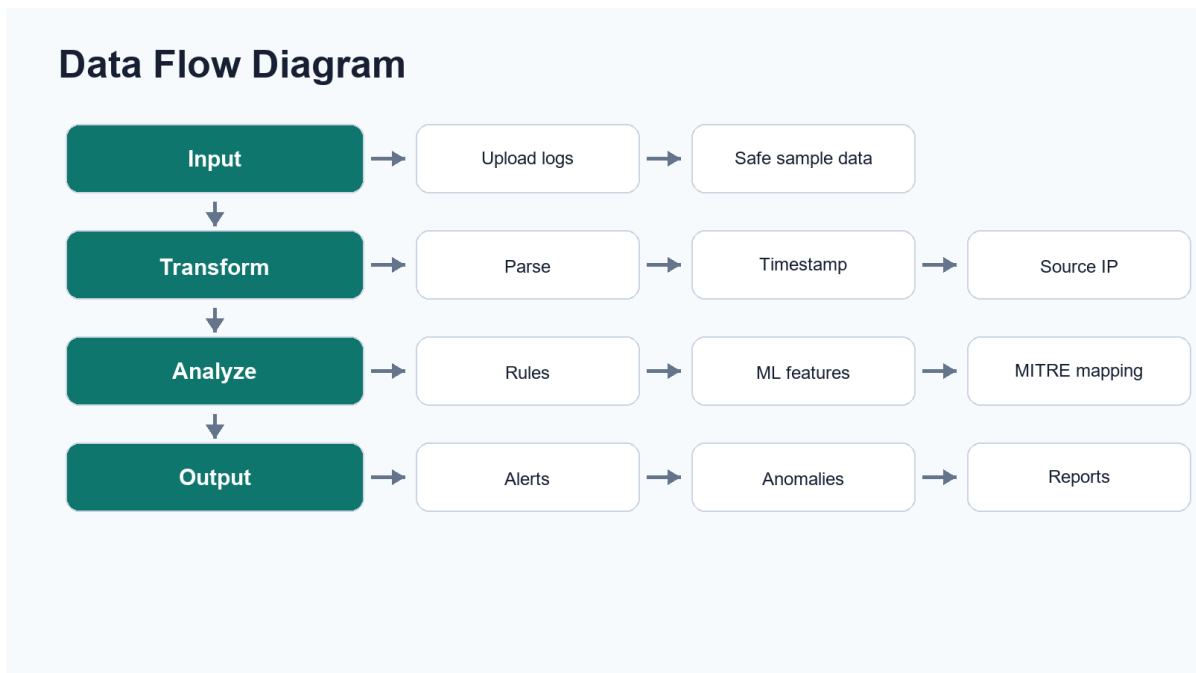


Fig. 2. SentinelLite data-flow diagram.

The parser layer produces a normalized table with timestamp, source file, log type, host, source IP, user, event type, method, path, status code, bytes sent, user agent, and raw message. The rule engine then evaluates decoded payloads and event windows. The anomaly layer groups source-IP activity into 15-minute windows and extracts numeric features for Isolation Forest.

The design keeps raw log text alongside normalized fields. This matters for forensics and viva defense: even if a parser does not understand every vendor-specific field, the analyst can still search and inspect the original evidence.

Threat Model

Threat	Detection/Control
Brute-force login	10-minute failed-login burst and success-after-failure detection.
Public web exploitation	SQL injection and path traversal probes mapped to T1190.
Network reconnaissance	Repeated 404, scanner user agent, and firewall DPT burst detection.
Endpoint execution	Suspicious PowerShell command-line detection.
Defense evasion	Windows audit-log clearing detection.

3.1 Data Model

Field	Purpose
timestamp	Timeline analysis and event-window aggregation.
source_ip	Primary entity for grouping attacks and anomaly windows.
event_type	Normalized category used by rules and charts.
raw_message	Original evidence retained for analyst review.
severity/risk_score	Triage priority for alerts.
mitre_technique	Threat-informed classification for reporting.

The data model is deliberately simple so that it remains explainable. The same normalized fields support charts, rule windows, anomaly features, and PDF reporting.

4. Implementation

SentinelLite is implemented as a modular Python application. The ``sentinellite.parsers`` module normalizes logs, ``sentinellite.rules`` produces explainable alerts, ``sentinellite.anomaly`` scores behavior windows, ``sentinellite.database`` stores run history and email settings, and ``app.py`` provides the dashboard.

```
features = [request_count, failed_login_count, error_count, unique_path_count, post_count, scanner_user_agent_count]
model = IsolationForest(n_estimators=150, contamination=0.15, random_state=42)
model.fit(StandardScaler().fit_transform(features))
```

The dashboard provides overview charts, alert filters, investigation expanders with related logs, anomaly scatter plots, log search, evaluation metrics, PDF report generation, and SMTP email digest sending.

4.1 Rule Engine Logic

The rule engine uses two kinds of logic. Row-level rules inspect individual log events for payloads such as SQL injection strings, path traversal, suspicious PowerShell execution, or Windows audit clearing. Windowed rules group events by source IP and time window to detect brute-force bursts, repeated 404 enumeration, scanner user agents, and port-scan behavior.

URL payloads are decoded before matching so that encoded strings such as ``%27%20or%201=1--`` are still detected. Scanner user-agent alerts are aggregated per source window to reduce noisy duplicate alerts.

4.2 Dashboard Workflow

The UI is designed as a small security operations console. The overview tab gives high-level posture, the alerts tab supports triage and related-log review, the anomaly tab shows ML scoring, the log explorer supports evidence search, the evaluation tab reports detection accuracy, and the report/email tabs support submission and operational output.

5. Security Analysis

Rules are intentionally transparent. Each alert includes severity, numeric risk score, confidence, attack phase, MITRE mapping, evidence, and response recommendation. This makes alerts defensible during viva and reduces the common problem of black-box alerting.

Rule	Severity	MITRE
Brute-force burst	High	T1110
Success after failures	Critical	T1078
SQL injection	High	T1190
Port scan pattern	High	T1046
PowerShell execution	High	T1059.001
Audit log cleared	Critical	T1070

The SMTP email module avoids storing passwords. Credentials are entered at send time, which reduces the risk of accidentally persisting secrets in SQLite. Uploaded logs are treated as text and are never executed.

5.1 Attack Scenarios Tested

Scenario	Evidence	Expected Alert
Credential attack	Multiple failed SSH and Windows logons	Authentication brute-force burst
Possible compromise	Successful login after repeated failures	Successful login after failures
Web exploitation	UNION SELECT and OR 1=1 payloads	SQL injection probe
Reconnaissance	Multiple DPT firewall blocks	Port scan pattern
Endpoint execution	PowerShell with bypass/encoded command	Suspicious PowerShell execution
Defense evasion	Windows Event ID 1102	Audit log cleared

These scenarios map directly to the brief's requirement for rule-based known-pattern detection and structured testing.

6. Results and Discussion

Alert Distribution



Fig. 3. Alert distribution across the safe sample dataset.

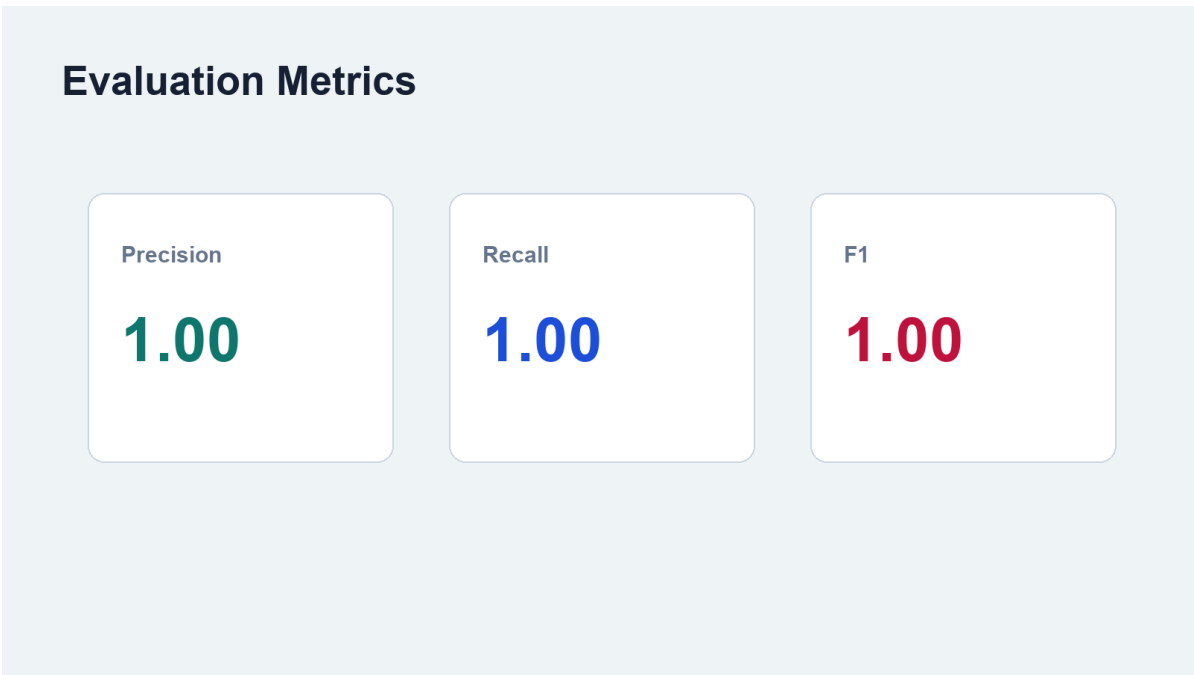


Fig. 4. Evaluation metrics on the labeled safe sample dataset.

Metric	Value
Parsed events	49
Alert groups	15
Anomaly windows	3
Precision	1.0

Metric	Value
Recall	1.0
F1 score	1.0

The results show that SentinelLite detects every labeled attack group in the controlled sample without generating extra alert groups. The perfect score reflects a small curated demonstration dataset, not a claim of production-grade generalization. The main value is that the methodology is testable, repeatable, and transparent.

6.1 Comparison with Existing Tools

Tool Type	Strength	SentinelLite Position
Enterprise SIEM	Scale, correlation, integrations	Too heavy for small labs; SentinelLite demonstrates cor
Log anomaly research tools	Benchmarking and algorithm comparison	Often less focused on live student demo workflow.
Manual log review	Simple and transparent	Slow and inconsistent; SentinelLite automates repeatab
IDS tools	Network-level detection	Complementary; SentinelLite focuses on log evidence.

SentinelLite is valuable because it is complete at the workflow level, even though it is intentionally smaller than enterprise tools.

6.2 Limitations

The current evaluation uses a small, controlled dataset. The system may miss attacks that do not produce visible logs, attacks hidden inside encrypted traffic before application logging, or slow attacks that stay below configured thresholds. Isolation Forest may flag rare but legitimate administrator behavior. These limitations are documented so the project remains technically honest.

7. Conclusion

SentinelLite satisfies the selected Tier A project requirements by providing a working lightweight SIEM with multi-source log ingestion, rule detections, unsupervised anomaly detection, email alerting, SQLite storage, dashboard visualization, testing evidence, and report generation. Future work should add scheduled ingestion, larger public datasets, Telegram alerts, incident ticket creation, and richer MITRE ATT&CK; reporting.

References

- [1] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92, 2006. Available: <https://csrc.nist.gov/pubs/sp/800/92/final>
- [2] F. T. Liu, K. M. Ting, and Z.-H. Zhou, Isolation Forest, Proc. IEEE ICDM, 2008. Available: <https://www.lamda.nju.edu.cn/publication/icdm08b.pdf>
- [3] M. Mantyla, Y. Wang, and J. Nyssola, LogLead: Fast and Integrated Log Loader, Enhancer, and Anomaly Detector, arXiv:2311.11809, 2023. Available: <https://arxiv.org/abs/2311.11809>
- [4] MITRE ATT&CK;, Brute Force, Technique T1110. Available: <https://attack.mitre.org/techniques/T1110/>
- [5] MITRE ATT&CK;, Network Service Discovery, Technique T1046. Available: <https://attack.mitre.org/techniques/T1046/>
- [6] MITRE ATT&CK;, Exploit Public-Facing Application, Technique T1190. Available: <https://attack.mitre.org/techniques/T1190/>
- [7] MITRE ATT&CK;, Command and Scripting Interpreter: PowerShell, T1059.001. Available: <https://attack.mitre.org/techniques/T1059/001/>
- [8] MITRE ATT&CK;, Indicator Removal, Technique T1070. Available: <https://attack.mitre.org/techniques/T1070/>
- [9] D. Basu and V. S. S. Yadav, Web Traffic Anomaly Detection Using Isolation Forest, Information, 2024. Available: <https://www.mdpi.com/2227-9709/11/4/83>
- [10] Practitioners' Expectations on Log Anomaly Detection, arXiv:2412.01066, 2024. Available: <https://arxiv.org/html/2412.01066v1>
- [11] System log anomaly detection based on contrastive learning and semantic representations, Scientific Reports, 2025. Available: <https://www.nature.com/articles/s41598-025-22208-7>

Appendix A: User Manual

Run ``python -m pip install -r requirements.txt``, then ``python -m streamlit run app.py``. Alternatively, on Windows run ``run_sentinellite.ps1``. Open <http://localhost:8501>, keep the safe sample logs enabled, and click Run SIEM Analysis.

Appendix B: GitHub Link Placeholder

GitHub repository link: To be added after the repository is created and pushed. The course brief requires commit history; this must be completed before final submission.